

Data Normalization: Breaking Tables Into the Proper Form

What Is Normalization?

Normalization is the process of breaking a denormalized table into multiple organized tables following specific rules called **Normal Forms**.

Each Normal Form has rules. Follow them, and your data becomes cleaner, faster, and easier to maintain. Violate them, and you inherit problems: duplication, inconsistency, insertion/deletion anomalies.

Think of it like organizing a messy desk into drawers. First, you separate papers from office supplies (1NF). Then, you organize papers by type (2NF). Then, by date within type (3NF). Each level of organization has a purpose.

Why Normalize?

Before we dive into rules, understand the cost of *not* normalizing.

Anomalies (Problems)

Insertion Anomaly

Can't insert data without including unrelated data.

Example: You want to add a new product to your Products table, but you can't—because your data isn't split yet. Products are still embedded in Transactions. You'd have to create a fake transaction just to record the product.

Update Anomaly

Change one fact, and you have to update it in multiple places or your data becomes inconsistent.

Example: Joan Roche (Customer 266019) moves to a new city. In your denormalized table, she appears 3 times. You update her city in row 1, forget row 2, leave row 3 as is. Now Joan lives in three different cities depending on which row you query.

Deletion Anomaly

Delete one row, and you lose unrelated data.

Example: Customer 759705 (Callisto Lo Duca) made 3 purchases. You delete all his transactions because he requested removal. Now the database has no record he ever existed—no history, no reference, nothing.

Normalization prevents all three.

The Normal Forms: A Progression

Normalization happens in stages. You apply rules, fix violations, move to the next stage. Higher normal forms are "more normalized."

```
Unnormalized
  ↓ Apply 1NF rules ↓
First Normal Form (1NF)
  ↓ Apply 2NF rules ↓
Second Normal Form (2NF)
  ↓ Apply 3NF rules ↓
Third Normal Form (3NF)
  ↓ Apply BCNF rules (optional) ↓
Boyce-Codd Normal Form (BCNF)
```

Most business databases stop at 3NF. BCNF is rare and usually unnecessary.

Stage 1: Unnormalized → First Normal Form (1NF)

The Problem: Repeating Groups

Unnormalized data has repeating groups: multiple values in a single cell.

Rule for 1NF:

- Each cell contains only a single value (atomic value)
- No repeating groups
- Each row is unique

Your Data: Is It 1NF?

Let's check. Your table has:

```
TransactionID | OrderNumber | LineItem | CustomerID | ProductID | Quantity | ...
3660001 | 366000 | 1 | 265598 | 1304 | 1 | ...
3660011 | 366001 | 1 | 1269051 | 1048 | 2 | ...
3660012 | 366001 | 2 | 1269051 | 2007 | 1 | ...
```

Each cell has one value. No repeating groups. **Your data is already 1NF. ✓**

Example of NOT 1NF:

```
✗ WRONG: Repeating groups
OrderID | CustomerName | Products | Quantities
```

```
366001 | Frank | 1048,2007 | 2,1
366002 | Joan | 1304,1048,2007 | 1,2,4
```

Products cell contains multiple comma-separated values.
Not atomic. Violates 1NF.

Fix: One row per product per order.

✓ CORRECT:

OrderID	CustomerName	ProductID	Quantity
366001	Frank	1048	2
366001	Frank	2007	1
366002	Joan	1304	1
366002	Joan	1048	2
366002	Joan	2007	4

Stage 2: 1NF → Second Normal Form (2NF)

The Problem: Partial Dependency

Partial dependency: A non-key column depends on only *part* of a composite key, not the whole key.

Rule for 2NF:

- Table must be in 1NF
- All non-key columns must depend on the *entire* primary key, not just part of it

Your Data: Check for 2NF Violations

Your Transactions table uses TransactionID as PK (single key, not composite).

```
TransactionID (PK) | OrderNumber | LineItem | CustomerID | ProductID | Quantity |
OrderDate | ...
```

Check each non-key column:

- Does OrderNumber depend on TransactionID? Yes. ✓
- Does Quantity depend on TransactionID? Yes. ✓
- Does OrderDate depend on TransactionID? Yes. ✓

All good. **Your Transactions table is 2NF.** ✓

But let's check your full denormalized table:

```
TransactionID | CustomerID | CustomerName | CustomerCity | ProductID | ProductName  
| ...
```

Does **CustomerName** depend on TransactionID? **No**. It depends on **CustomerID**.

Does **ProductName** depend on TransactionID? **No**. It depends on **ProductID**.

Your full table violates 2NF. ❌

How to Fix: Extract Dependent Columns

Identify columns that depend on something other than the primary key, and move them to a new table.

Columns dependent on CustomerID:

```
CustomerID, CustomerName, CustomerGender, CustomerCity,  
CustomerStateCode, CustomerState, CustomerZip, CustomerCountry,  
CustomerContinent, CustomerDOB
```

Create Customers Table:

```
Customers  
├ CustomerID (PK)  
├ CustomerGender  
├ CustomerName  
├ CustomerCity  
├ CustomerStateCode  
├ CustomerState  
├ CustomerZip  
├ CustomerCountry  
├ CustomerContinent  
└ CustomerDOB
```

Columns dependent on ProductID:

```
ProductID, ProductName, ProductBrand, ProductColor,  
ProductCost, ProductPrice, ProductSubcategoryID, ProductCategoryID
```

Create Products Table:

Products

- └ ProductID (PK)
- └ ProductName
- └ ProductBrand
- └ ProductColor
- └ ProductCost
- └ ProductPrice
- └ ProductSubcategoryID (FK)
- └ ProductCategoryID (FK)

Columns dependent on StoreID:

StoreID, StoreCountry, StoreState, StoreSqMeters, StoreOpenDate

Create Stores Table:

Stores

- └ StoreID (PK)
- └ StoreCountry
- └ StoreState
- └ StoreSqMeters
- └ StoreOpenDate

What's left in Transactions:

Transactions

- └ TransactionID (PK)
- └ OrderNumber
- └ LineItem
- └ OrderDate
- └ DeliveryDate
- └ Quantity
- └ CustomerID (FK)
- └ StoreID (FK)
- └ ProductID (FK)

All non-key columns now depend on TransactionID only. 2NF ✓

Stage 3: 2NF → Third Normal Form (3NF)

The Problem: Transitive Dependency

Transitive dependency: A non-key column depends on another non-key column, not on the

primary key.

Rule for 3NF:

- Table must be in 2NF
- No non-key column should depend on another non-key column

Example of Transitive Dependency

In your Products table:

ProductID (PK)	ProductName	ProductSubcategoryID	ProductSubcategoryName	ProductCategoryID	ProductCategoryName
1304	Contoso Lens Adapter	0406	Cameras & Camcorders Accessories	04	Cameras and camcorders
1048	A. Datum SLR Camera	0402	Digital SLR Cameras	04	Cameras and camcorders
2007	Fabrikam Microwave	0803	Microwaves	08	Home Appliances

Check dependencies:

- **ProductName** depends on **ProductID**? Yes. ✓
- **ProductSubcategoryID** depends on **ProductID**? Yes. ✓
- **ProductSubcategoryName** depends on **ProductID**? **No**. It depends on **ProductSubcategoryID**. ✗
- **ProductCategoryID** depends on **ProductID**? Yes. ✓
- **ProductCategoryName** depends on **ProductID**? **No**. It depends on **ProductCategoryID**. ✗

Violation: **ProductSubcategoryName** and **ProductCategoryName** create transitive dependencies.

How to Fix: Extract Hierarchies

Move columns that depend on non-key columns to new tables.

Create ProductSubcategories Table:

ProductSubcategories
├─ ProductSubcategoryID (PK)
├─ ProductSubcategoryName
└─ ProductCategoryID (FK)

Create ProductCategories Table:

```
ProductCategories
├─ ProductCategoryID (PK)
└─ ProductCategoryName
```

Update Products Table:

```
Products
├─ ProductID (PK)
├─ ProductName
├─ ProductBrand
├─ ProductColor
├─ ProductCost
├─ ProductPrice
└─ ProductSubcategoryID (FK) → ProductSubcategories
```

(Remove ProductSubcategoryName and ProductCategoryID/Name from Products—they now come through the subcategory relationship)

Now all non-key columns depend directly on the PK. 3NF ✓

Stage 4: 3NF → BCNF (Boyce-Codd Normal Form)

The Problem: When PK Has Multiple Columns

BCNF Rule: Every determinant must be a candidate key.

This is complex and rarely needed in practice. We'll skip it for your model since your PKs are straightforward.

When to care about BCNF: When you have complex composite keys with overlapping attributes. Rare.

Your Denormalized Table → Normalized Model

Let's walk through the full process using your actual data.

Step 1: Start (Unnormalized)

```
One mega table with 30 columns:
TransactionID | OrderNumber | LineItem | OrderDate | DeliveryDate | Quantity |
CustomerID | CustomerGender | CustomerName | CustomerCity | CustomerStateCode |
CustomerState |
CustomerZip | CustomerCountry | CustomerContinent | CustomerDOB |
StoreID | StoreCountry | StoreState | StoreSqMeters | StoreOpenDate |
```

```
ProductID | ProductName | ProductBrand | ProductColor | ProductCost | ProductPrice |
|
ProductSubcategoryID | ProductSubcategory | ProductCategoryID | ProductCategory
```

Problems:

- Duplication (Customer info repeated per transaction)
- Update anomalies (change Joan's address, must update multiple rows)
- Insertion anomalies (can't add product without transaction)

Step 2: Apply 1NF

Check: Are all cells atomic? Are rows unique?

Result: Already 1NF. ✓ No repeating groups.

Step 3: Apply 2NF

Identify columns that don't depend on the primary key. Extract them.

Violations found:

- Customer columns depend on CustomerID, not TransactionID
- Product columns depend on ProductID, not TransactionID
- Store columns depend on StoreID, not TransactionID

Create separate tables:

```
Transactions (fact):
  TransactionID (PK) | OrderNumber | LineItem | OrderDate | DeliveryDate |
  Quantity |
  CustomerID (FK) | StoreID (FK) | ProductID (FK)

Customers (dimension):
  CustomerID (PK) | CustomerGender | CustomerName | CustomerCity |
  CustomerStateCode |
  CustomerState | CustomerZip | CustomerCountry | CustomerContinent | CustomerDOB

Stores (dimension):
  StoreID (PK) | StoreCountry | StoreState | StoreSqMeters | StoreOpenDate

Products (dimension):
  ProductID (PK) | ProductName | ProductBrand | ProductColor | ProductCost |
  ProductPrice |
  ProductSubcategoryID | ProductCategoryID
```

Result: 2NF ✓

Step 4: Apply 3NF

Check for transitive dependencies (non-key columns depending on other non-key columns).

Violations found in Products table:

- ProductSubcategory depends on ProductSubcategoryID, not ProductID
- ProductCategory depends on ProductCategoryID, not ProductID

Extract hierarchies:

```
ProductSubcategories (dimension):
  ProductSubcategoryID (PK) | ProductSubcategoryName | ProductCategoryID (FK)

ProductCategories (dimension):
  ProductCategoryID (PK) | ProductCategoryName

Products (updated):
  ProductID (PK) | ProductName | ProductBrand | ProductColor | ProductCost |
  ProductPrice |
  ProductSubcategoryID (FK)
```

Remove ProductCategoryID from Products (it's accessible through ProductSubcategories).

Result: 3NF ✓

Final Normalized Model

FACT TABLE (the transaction event):

```
Transactions
├─ TransactionID (PK)
├─ OrderNumber
├─ LineItem
├─ OrderDate
├─ DeliveryDate
├─ Quantity
├─ CustomerID (FK) → Customers
├─ StoreID (FK) → Stores
└─ ProductID (FK) → Products
```

DIMENSION TABLES (the reference data):

Customers

- └ CustomerID (PK)
- └ CustomerGender
- └ CustomerName
- └ CustomerCity
- └ CustomerStateCode
- └ CustomerState
- └ CustomerZip
- └ CustomerCountry
- └ CustomerContinent
- └ CustomerDOB

Stores

- └ StoreID (PK)
- └ StoreCountry
- └ StoreState
- └ StoreSqMeters
- └ StoreOpenDate

Products

- └ ProductID (PK)
- └ ProductName
- └ ProductBrand
- └ ProductColor
- └ ProductCost
- └ ProductPrice
- └ ProductSubcategoryID (FK) → ProductSubcategories

ProductSubcategories

- └ ProductSubcategoryID (PK)
- └ ProductSubcategoryName
- └ ProductCategoryID (FK) → ProductCategories

ProductCategories

- └ ProductCategoryID (PK)
- └ ProductCategoryName

Verification: Did It Work?

Check for Anomalies

Insertion Anomaly: Can I add a new product without a transaction?

- **Before:** No. Products were only in transactions.

- **After:** Yes. Insert into Products table anytime. ✓

Update Anomaly: Can I update customer data in one place?

- **Before:** No. Joan appears 3 times. Had to update 3 rows.
- **After:** Yes. Update Customers table once. All references automatically point to updated data. ✓

Deletion Anomaly: Can I delete a transaction without losing customer info?

- **Before:** No. Delete all transactions, customer disappears.
- **After:** Yes. Delete transactions. Customers table still exists with complete history. ✓

Check Dependencies

No partial dependencies (2NF)?

- Transactions: All non-key columns depend on TransactionID only. ✓
- Customers: All non-key columns depend on CustomerID only. ✓
- Products: All non-key columns depend on ProductID only. ✓

No transitive dependencies (3NF)?

- No non-key column depends on another non-key column. ✓

Summary: The Normalization Checklist

Form	Rule	Your Data
1NF	Atomic values, no repeating groups	✓ Passes
2NF	All non-key columns depend on entire PK	X Violated (extracted dimensions) → ✓ Fixed
3NF	No non-key column depends on another non-key column	X Violated (category hierarchy) → ✓ Fixed
BCNF	Every determinant is a candidate key	✓ Not needed (simple PKs)

Why 3NF Is Usually Enough

Higher normal forms (BCNF, 4NF, 5NF) exist but are rarely needed in business analytics.

3NF gives you:

- Elimination of duplication
- Prevention of anomalies
- Reasonable performance
- Easy maintenance

BCNF, 4NF, 5NF add:

- Marginal additional data consistency
- Increased complexity
- More tables, more joins, potentially slower queries

For your model and 95% of business data, 3NF is the sweet spot.

From Normalization to Power Query

Now that you understand *why* normalization exists and *how* to identify violations, building the normalized model in Power Query is straightforward:

1. Start with your denormalized table
2. Create a Customers query (extract and deduplicate Customer columns)
3. Create a Stores query
4. Create a ProductCategories query
5. Create a ProductSubcategories query
6. Create a Products query
7. Clean your Transactions query (keep only transaction-specific columns + FK columns)
8. Load all six queries to Power BI
9. Build 1:N relationships between fact and dimensions
10. Analyze

The normalization rules you just learned guide every decision.

Key Takeaways

Normalization is about elimination:

- Eliminate duplication
- Eliminate anomalies
- Eliminate ambiguity

Normal Forms are a spectrum:

- 1NF: Atomic values
- 2NF: No partial dependencies
- 3NF: No transitive dependencies

Your denormalized table violated 2NF and 3NF:

- 2NF violation: Customer/Product/Store columns didn't depend on the transaction PK
- 3NF violation: Category columns didn't depend directly on the product PK

Your fixed model is 3NF:

- 6 tables (1 fact, 5 dimensions)
- No anomalies
- No duplication
- Fast queries
- Easy to maintain

This is what professional data models look like. Master this pattern, and you can normalize any denormalized table.

Practice Question

Look at your data. You have `CustomerStateCode` and `CustomerState`.

Both exist in the Customers table. Are they redundant? Should one be removed?

Answer: Semantically, yes—they represent the same thing (Canadian province, US state). In practice, you'd keep StateCode as the PK reference (short, fast to join) and derive State from a lookup table. Or vice versa. But both in one table? Analyze the use case.

For your module, keeping both is fine. It's not a normalization violation (both depend on CustomerID). It's a design choice. Good data models optimize for use, not purity.